클릭 한 번에 삼각형 10만 개를 훑고 있었다

Minecraft Dungeons 모작의 맵 피킹 최적화

C++ / DirectX 11 개인 프로젝트 · 전수 탐색 → BVH

3줄 요약

- 맵 메시 하나가 삼각형 수천 개 덩어리 → 클릭할 때마다 전부 검사하고 있었음
- 메시 안쪽 탐색 단계만 BVH로 교체 (기존 피킹 입출력은 그대로)
- 삼각형 검사량 99.77% 감소 / 메시 탐색 구간 시간 약 12.35배 단축
- 같은 결과가 나오는지 14만 9천 쌍으로 대조해서 확인

문제의 시작은 맵 제작 방식이었다

- 개발 기간이 짧아서 맵을 직접 모델링할 수 없었음
- Minecraft Dungeons 맵 → Minecraft로 내보내기 → 외부 도구로 FBX 추출
- 이 경로는 오브젝트 단위로 메시를 잘라주지 않음
- 결과: 메시 하나가 큰 덩어리로 생성, 그 안에 삼각형 수천 개

증상: 누른 채로 끌면 느려진다

- 특정 구역에서 왼쪽 버튼을 누른 채 이동하면 플레이어 이동이 눈에 띄게 느려짐
- 클릭 한 번만 하면 멀쩡함 → 단발 테스트로는 안 잡힘
- 이유: 버튼을 누르고 있는 동안 매 프레임 맵 피킹을 "새로" 요청

```
// Client/Private/State_Player.cpp:106
_bool CState_Player::Change_State_To_Walk()
{
    if (m_pGameInstance->Get_Key(VK_LBUTTON) && !bMouseClickLock)
    {
        ...
        if (m_pGameInstance->Picked_Model(fWorldPickedPos, mapTag,
            currentLevel, TEXT("Layer_BackGround")))   // ← 누르고 있는 동안 계속 호출
        {
            m_pPlayer->Set_NextPosition(fWorldPickedPos);
            m_pPlayer->Change_State(PLAYER_STATE::WALK);
        }
    }
}
```

원인 추적: 프로파일러로 구간 쪼개기

Visual Studio 프로파일러로 피킹 구간을 세분화하고 코드를 따라감

```
State_Player::Change_State_To_Walk
  └ CPicking::Picked_Model                화면 좌표 → 월드 광선
      └ CModel::Picking_Model              월드 → 로컬 변환, 메시 루프
          ├ CMesh::Check_BoundingBox_AABB   메시 외곽 AABB  — 싸다
          └ CMesh::Picking_In_Mesh          삼각형 전수 검사 — 비싸다
```

- 외곽 AABB는 메시당 slab 검사 1번이라 문제 없음
- 병목은 "AABB를 통과한 메시 안에서 삼각형을 전부 훑는" 단계

병목: 메시 안의 모든 삼각형을 검사

```
// Engine/Private/Mesh.cpp:614  CMesh::Picking_In_Mesh
for (_uint i = 0; i < m_iNumIndices; i += 3)   // ← 메시의 모든 삼각형
{
    _float3 vA = m_pVertices[m_pIndices[i + 0]];
    _float3 vB = m_pVertices[m_pIndices[i + 1]];
    _float3 vC = m_pVertices[m_pIndices[i + 2]];

    _vector v0 = XMLoadFloat3(&vA);
    _vector v1 = XMLoadFloat3(&vB);
    _vector v2 = XMLoadFloat3(&vC);

    if (TriangleTests::Intersects(vOrigin, vDir, v0, v1, v2, fDist))
    {
        if (fDist < fMinDist)       // 가장 가까운 적중만 남김
        {
            fMinDist = fDist;
            XMStoreFloat3(&vOutLocalPickedPos, vOrigin + vDir * fDist);
            fOutDist = fDist;  bHit = true;
        }
    }
}
```

- 광선이 스치기만 해도 전부 검사. 후보 메시가 여러 개면 그만큼 곱해짐

숫자로 보면

상황	한 번 피킹할 때 검사한 삼각형
로비 / 누른 채 이동	약 21,598개
로비 / 후보 메시가 많은 좌표	약 97,982개

- 이걸 버튼 누르고 있는 동안 매 프레임 반복
- 측정 기준: Release 재생 1,800프레임 × 3회 평균

1차 대응: Blender로 메시 쪼개기

- 당시엔 일정상 BVH를 직접 구현·검증할 여유가 없었음
- Blender에서 큰 메시를 잘라서 메시당 삼각형 수를 줄임
- 외곽 AABB가 더 잘 걸러주니까 효과는 있었음
- 다만 근본 구조는 그대로 — "통과한 메시 안은 여전히 전수 검사"

※ 뒤에 나올 성능 수치는 전부 "메시를 이미 쪼갠 상태"에서 탐색 방식만 바꾼 비교입니다.

뭘 쓸까: 균일 그리드 / 옥트리 / BVH

방식	자르는 대상	이 문제에서의 성질
균일 그리드	공간을 격자로	구현은 제일 쉬움. 지형처럼 밀도가 들쭉날쭉하면 빈 칸과 과밀 칸이 동시에 생김
옥트리	공간을 8등분씩	삼각형이 칸 경계에 걸치면 중복 등록하거나 잘라야 함. 칸 경계가 기하와 무관
BVH	삼각형(=물체)을	삼각형이 한 리프에만 들어감(중복 없음). 박스가 실제 기하에 밀착

왜 BVH를 골랐나

기존 코드에 이미 있던 것

① 월드 → 로컬 광선 변환
② 메시별 외곽 AABB 컷

→ 새로 만들 게 "공간 분할 체계 전체"가 아니라
③ "큰 메시 안에서 삼각형 찾는 단계" 하나였음

BVH가 맞았던 이유

- 삼각형을 중복 없이 한 리프에만 넣음 → 인덱스 관리가 단순
- 노드 박스가 실제 삼각형에 밀착 → 빈 공간을 덜 탐색
- 반환 규약(로컬 적중 위치 + 거리)을 기존과 똑같이 유지 가능
- 맵 로딩 때 한 번만 짓고 계속 재사용

자료구조: 노드 배열 + 삼각형 ID 배열

```
// Engine/Public/Mesh.h:12
struct BVH_Node
{
    _float3 boundingMin, boundingMax;  // 이 노드가 담당하는 삼각형 전체를 감싸는 AABB
    _int    iLeft, iRight;             // 자식 인덱스 (둘 다 -1이면 리프)
    _int    iTriStart, iTriCount;      // m_TriangleIDs의 담당 구간
};

// Engine/Public/Mesh.h:97
vector<BVH_Node> m_BVHNodes;      // 평탄 배열. 포인터 대신 인덱스로 자식 참조
vector<_uint>    m_TriangleIDs;   // 삼각형 ID. "순서"만 재배치됨
```

- 노드는 정점/인덱스를 복사하지 않고 구간만 가리킴
- 접근 경로: 구간 위치 → 삼각형 ID → ID×3 → m_pIndices → m_pVertices
- 렌더링용 인덱스 버퍼는 손대지 않음 (중요)

구축: 맵 로딩 때 한 번

```
// Engine/Private/Mesh.cpp:315
void CMesh::Build_BVH()
{
    const _uint iTriCount = m_iNumIndices / 3;

    m_TriangleIDs.resize(iTriCount);
    for (_uint i = 0; i < iTriCount; ++i)
        m_TriangleIDs[i] = i;          // 0, 1, 2, ... 로 초기화

    m_BVHNodes.clear();
    Build_BVH_Node(0, iTriCount);      // 루트부터 재귀 분할
}

// Client/Private/LoungeMap.cpp:52      ← 맵 오브젝트 Initialize 안에서 1회
m_pModelCom->Build_BVH();
// Client/Private/SoggySwampMap.cpp:54
m_pModelCom->Build_BVH();
```

어떻게 자르나

분할축　　삼각형 중심들이 가장 넓게 퍼진 축을 고름
분할 위치　그 축 기준으로 개수 중앙값 (nth_element)
리프 조건　삼각형 8개 이하　또는　중심들이 거의 같은 자리에 몰려 있을 때

```
// Engine/Private/Mesh.cpp:514  — 정렬이 아니라 "중앙값 자리만" 맞춤 → O(n)
_int CMesh::Partition_Median(_int iStart, _int iTriCount, _int iAxis)
{
    const _int mid = iStart + iTriCount / 2;

    auto key = [&](const _uint triangleID) -> _float {
        const _float3 c = Compute_Triangle_Centroid(triangleID);
        return (iAxis == 0) ? c.x : (iAxis == 1 ? c.y : c.z);
    };

    std::nth_element(
        m_TriangleIDs.begin() + iStart,
        m_TriangleIDs.begin() + mid,
        m_TriangleIDs.begin() + (iStart + iTriCount),
        [&](const _uint a, const _uint b) { return key(a) < key(b); });

    return mid;
}
```

리프에서 멈추는 조건

```
// Engine/Private/Mesh.cpp:556
bool CMesh::Should_Stop_Splitting(_int count, const _float3& centroidExtent) const
{
    constexpr int LEAF_TRI_MAX = 8;
    if (count <= LEAF_TRI_MAX) return true;

    // 중심이 거의 같으면 더 쪼개도 반으로 안 나뉘고 재귀만 깊어짐
    const float eps = 1e-6f;
    if (centroidExtent.x < eps && centroidExtent.y < eps && centroidExtent.z < eps)
        return true;

    return false;
}
```

- 두 번째 조건은 "중심이 똑같은 삼각형 뭉치"에서 무한히 깊어지는 걸 막는 안전장치
- 리프가 8개 이하라 트리 깊이는 대략 $\log_2$(삼각형 수 / 8)

탐색: 방문할 노드를 스택에 쌓는다

```
// Engine/Private/Mesh.cpp:682
// 루트 박스를 못 맞히면 바로 끝
if (!Ray_Intersects_AABB(localMousePos, localMouseRay,
    m_BVHNodes[0].boundingMin, m_BVHNodes[0].boundingMax, tminRoot, tmaxRoot))
    return false;

std::vector<_int> stack;      // 함수 재귀 대신 "앞으로 볼 노드 번호" 목록
stack.reserve(64);
stack.push_back(0);           // 루트

while (!stack.empty())
{
    const _int nodeIdx = stack.back();
    stack.pop_back();
    const BVH_Node& node = m_BVHNodes[nodeIdx];

    _float tmin = 0.f, tmax = FLT_MAX;
    if (!Ray_Intersects_AABB(localMousePos, localMouseRay,
        node.boundingMin, node.boundingMax, tmin, tmax))
        continue;             // 박스를 안 맞히면 이 가지 통째로 버림
    ...
}
```

핵심 두 가지: 가까운 쪽 먼저, 그리고 가지치기

```
// ① 가까운 자식 먼저 — Engine/Private/Mesh.cpp:730
_bool lhit = Ray_Intersects_AABB(..., m_BVHNodes[left ].boundingMin, ..., lmin, lmax);
_bool rhit = Ray_Intersects_AABB(..., m_BVHNodes[right].boundingMin, ..., rmin, rmax);

if (lhit && rhit)
{
    // 스택은 LIFO → "먼 쪽을 먼저 push"하면 가까운 쪽이 먼저 pop된다
    if (lmin < rmin) { stack.push_back(right); stack.push_back(left); }
    else             { stack.push_back(left);  stack.push_back(right); }
}

// ② 가지치기 — Engine/Private/Mesh.cpp:708
// 이미 더 가까운 적중을 찾았다면, 그보다 먼 박스는 볼 필요가 없다
if (hit && tmin > bestDist)
    continue;
```

- 가까운 쪽부터 보면 bestDist가 빨리 줄어듦 → ②의 가지치기가 더 많이 걸림
- 둘은 세트로 동작 (순서 → 더 센 가지치기)

리프에 도착하면? 거기선 전수 검사 (단, 최대 8개)

```
// Engine/Private/Mesh.cpp:776
_bool CMesh::Intersect_Leaf_Triangles(_int triStart, _int triCount,
    const _vector& vOrigin, const _vector& vDir,
    _float& ioBestDist, _float3& outPicked) const
{
    _bool hit = false;
    for (_int k = 0; k < triCount; ++k)
    {
        const _uint triId = m_TriangleIDs[triStart + k];   // ID 배열 경유
        const _uint base  = triId * 3;
        ...
        if (TriangleTests::Intersects(vOrigin, vDir, v0, v1, v2, dist))
        {
            if (dist < ioBestDist)      // 첫 적중에서 멈추지 않음
            {
                ioBestDist = dist;      // 공유 상태 → 다음 노드 가지치기에 바로 반영
                XMStoreFloat3(&outPicked, vOrigin + vDir * dist);
                hit = true;
            }
        }
    }
    return hit;
}
```

- bestDist를 참조로 공유 → 방문 순서와 무관하게 최종 결과는 최근접 적중. "빠르게"가 "대충"이 아님

삽질: vector가 이사 가면서 터질 뻔한 버그

- 부모 노드를 먼저 push_back → 자식을 재귀로 생성
- 재귀 도중 push_back이 용량을 넘기면 vector가 재할당(= 메모리 이사)
- 그러면 대입 대상으로 잡아둔 부모 요소의 위치가 무효가 될 수 있음

```
// 수정 전
m_BVHNodes[myIndex].iLeft  = Build_BVH_Node(iStart, mid - iStart);
m_BVHNodes[myIndex].iRight = Build_BVH_Node(mid, (iStart + iTriCount) - mid);

// 수정 후 — Engine/Private/Mesh.cpp:434
// 재귀를 먼저 끝내고, 그 다음에 부모 요소를 다시 조회해서 대입
const _int leftIndex  = Build_BVH_Node(iStart, mid - iStart);
const _int rightIndex = Build_BVH_Node(mid, (iStart + iTriCount) - mid);
m_BVHNodes[myIndex].iLeft  = leftIndex;
m_BVHNodes[myIndex].iRight = rightIndex;
```

"빨라졌다"보다 먼저 "같은 답이 나오나"

- --compare-picking 모드: 같은 메시, 같은 로컬 광선으로 두 방식을 모두 실행해서 결과를 대조
- 캐시 편향 줄이려고 호출 순서를 홀짝으로 교대
- 적중 여부 / 거리 / 위치를 허용오차와 비교

```
// Engine/Private/Model.cpp:211
// Alternate order so one implementation is not always second with warm caches.
static unsigned long long sequence = 0;
if ((sequence++ & 1) == 0) { brute(); bvh(); }
else                       { bvh(); brute(); }

// Engine/Private/Model.cpp:227   (슬라이드용 축약 — 원본엔 isfinite 검사도 함께)
const double tolerance = 1e-4 * (1.0 + std::fabs(double(fOutDist)));
mismatch = sample.maxPositionError > tolerance ||
           std::fabs(double(fOutDist) - bvhDistance) > tolerance;
```

정확성 검증 결과

항목	결과
대조한 동일 메시·광선 쌍	149,512쌍
허용오차 초과 결과 불일치	0
최대 적중 위치 차이	0.000009344 (로컬 좌표)
BVH 미구축 fallback 발생	0
고정 픽스처 128광선 회귀	Debug / Release 모두 통과

결과 ① 삼각형 검사량 99.77% 감소

실제 플레이 캡처 (로비, 동일 메시·광선 82,088쌍)

	전수 검사	BVH
삼각형 검사 총량	157,028,988	353,752 (-99.77%)
질의당 삼각형 검사	약 1,913개	약 4.3개
질의당 방문 노드	—	약 6.7개

결과 ② 메시 탐색 구간 약 12.35배 단축

같은 캡처, 계측된 메시 내부 탐색 구간

	전수 검사	BVH
메시 쿼리 1회 평균	0.012466 ms	0.001009 ms
피킹 있던 프레임당 메시 탐색 합계	0.296108 ms	0.023972 ms

→ 프레임당 약 0.272 ms 절약 / 구간 시간 비율 약 12.35배

결과 ③ 한 방식만 켜고 돌린 플레이 비용

Release, 입력·광선을 녹화해서 똑같이 재생, 조건당 3회 반복 / 피킹 호출 1회 평균

구간	전수 검사	BVH
로비 / 누른 채 이동	0.152837 ms	0.039733 ms
로비 / 후보 많은 좌표	0.673240 ms	0.082162 ms
늪지 / 누른 채 이동	0.112275 ms	0.026763 ms
늪지 / 후보 많은 좌표	0.177142 ms	0.048681 ms
호출당 삼각형 검사 (로비 누른 이동)	21,598	99
호출당 삼각형 검사 (로비 후보 많음)	97,982	319

- 두 모드의 후보 메시 수가 완전히 동일 → 같은 광선, 같은 작업량으로 비교된 것

이 숫자가 "아닌" 것

✕ 이렇게 말하면 틀림	○ 정확하게는
게임 FPS가 12배 빨라졌다	메시 내부 탐색 구간 시간 비율이 약 12.35배
0.27 프레임을 절약했다	프레임당 약 0.272 밀리초를 절약했다
클릭부터 화면에 보이기까지의 지연을 쟀다	CPU에서 입력 분기 ~ 이동 목표 설정까지의 구간 (재생 모드라 카메라 역투영은 건너뜀)
옥트리보다 BVH가 빨랐다	옥트리는 구현도 측정도 안 함. 전수 검사 대비 효과만 측정
메시 쪼개기 효과까지 포함된 숫자다	이미 쪼갠 메시에서 탐색 방식만 바꾼 비교

정리

- 맵 제작 방식 때문에 메시가 덩어리로 나왔고, 피킹이 그 안을 전부 훑고 있었음
- 기존 경로(로컬 광선 변환 + 메시 AABB)는 그대로 두고 안쪽 탐색만 BVH로 교체
- 삼각형 검사량 99.77% 감소, 메시 탐색 구간 약 12.35배 단축
- 정확성은 14만 9천 쌍 대조 + 128광선 회귀로 먼저 확인하고 성능을 봤음

가져갈 것

- 병목은 "느린 함수"가 아니라 "불필요하게 많이 도는 루프"인 경우가 많다
- 최적화는 같은 답이 나오는지 확인하는 장치를 먼저 만들고 시작하는 게 편하다